

✓ Labsheet 10

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

from scipy.stats import pearsonr
```

```
uploaded = files.upload()

file_name = list(uploaded.keys())[0]

df = pd.read_csv(file_name)

print("Dataset loaded successfully!")
print("Shape of dataset:", df.shape)
```

Choose Files House_Pric...Sample.csv
House_Price_Prediction_Sample.csv(text/csv) - 1333 bytes, last modified: 9/28/2026 - 100% done
 Saving House_Price_Prediction_Sample.csv to House_Price_Prediction_Sample (1).csv
 Dataset loaded successfully!
 Shape of dataset: (30, 11)

```
print("\nFirst 5 Records:")
display(df.head())

print("\nColumn Names:")
print(df.columns.tolist())

print("\nDataset Information:")
df.info()
```

First 5 Records:

	Id	OverallQual	OverallCond	GrLivArea	TotalBsmtSF	1stFlrSF	GarageArea	FullBath	BedroomAbvGr	YearBuilt	SalePrice
0	1	7	5	1710	856	856	548	2	3	2003	208500
1	2	6	8	1262	1262	1262	460	2	3	1976	181500
2	3	7	5	1786	920	920	608	2	3	2001	223500
3	4	7	5	1717	756	961	642	1	3	1915	140000
4	5	8	5	2198	1145	1145	836	2	4	2000	250000

Column Names:

['Id', 'OverallQual', 'OverallCond', 'GrLivArea', 'TotalBsmtSF', '1stFlrSF', 'GarageArea', 'FullBath', 'BedroomAbvGr', 'YearBuilt', 'SalePrice']

Dataset Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 30 entries, 0 to 29

Data columns (total 11 columns):

#	Column	Non-Null Count	Dtype
0	Id	30 non-null	int64
1	OverallQual	30 non-null	int64
2	OverallCond	30 non-null	int64
3	GrLivArea	30 non-null	int64
4	TotalBsmtSF	30 non-null	int64
5	1stFlrSF	30 non-null	int64
6	GarageArea	30 non-null	int64
7	FullBath	30 non-null	int64
8	BedroomAbvGr	30 non-null	int64
9	YearBuilt	30 non-null	int64
10	SalePrice	30 non-null	int64

dtypes: int64(11)

memory usage: 2.7 KB

```
# 4. CHECK MISSING VALUES
# =====

print("\nMissing Values:")
print(df.isnull().sum())

# =====
# 5. CHECK DUPLICATES
# =====

print("\nNumber of Duplicate Rows:")
print(df.duplicated().sum())

# Remove duplicates
df = df.drop_duplicates()

print("Shape after removing duplicates:", df.shape)

# =====
# 6. BASIC STATISTICAL ANALYSIS
# =====

print("\nStatistical Summary:")
display(df.describe())
```

Missing Values:

```
Id          0
OverallQual 0
OverallCond 0
GrLivArea   0
TotalBsmtSF 0
1stFlrSF    0
GarageArea   0
FullBath     0
BedroomAbvGr 0
YearBuilt    0
SalePrice    0
dtype: int64
```

Number of Duplicate Rows:

```
0
```

Shape after removing duplicates: (30, 11)

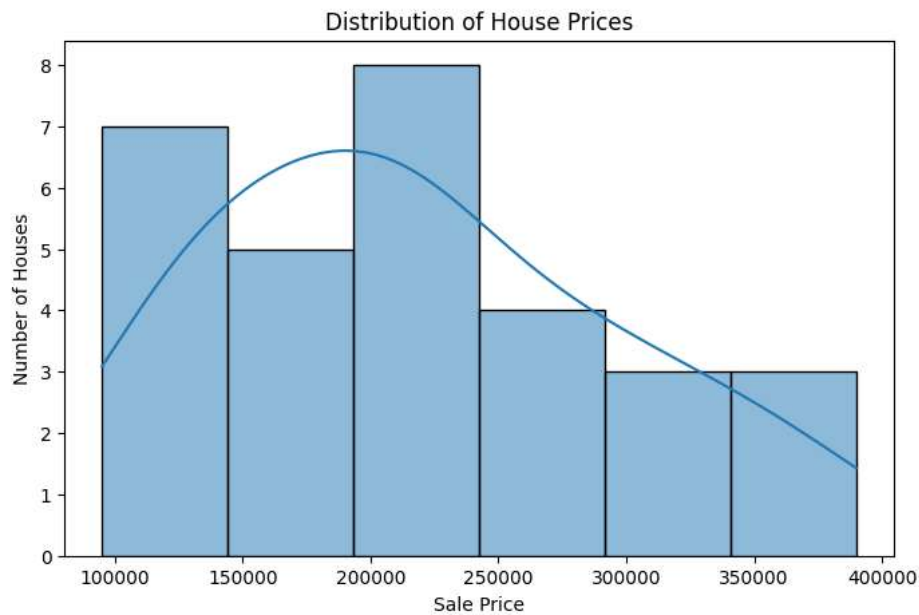
Statistical Summary:

	Id	OverallQual	OverallCond	GrLivArea	TotalBsmtSF	1stFlrSF	GarageArea	FullBath	BedroomAbvGr	YearBuilt
count	30.000000	30.000000	30.000000	30.000000	30.000000	30.000000	30.000000	30.000000	30.000000	30.000000
mean	15.500000	6.866667	5.400000	1707.066667	1071.333333	1092.033333	600.633333	1.866667	3.000000	1984.800000
std	8.803408	1.382984	0.770132	403.835912	272.153777	252.081060	193.041801	0.628810	0.643268	25.496991
min	1.000000	4.000000	5.000000	980.000000	650.000000	750.000000	205.000000	1.000000	2.000000	1915.000000
25%	8.250000	6.000000	5.000000	1412.500000	867.000000	903.000000	480.000000	1.250000	3.000000	1970.750000
50%	15.500000	7.000000	5.000000	1702.000000	1020.000000	1032.000000	604.000000	2.000000	3.000000	1997.500000
75%	22.750000	8.000000	5.750000	1972.500000	1237.500000	1237.500000	727.500000	2.000000	3.000000	2003.000000
max	30.000000	9.000000	8.000000	2500.000000	1686.000000	1694.000000	950.000000	3.000000	4.000000	2010.000000

```
plt.figure(figsize=(8,5))

sns.histplot(df["SalePrice"], kde=True)

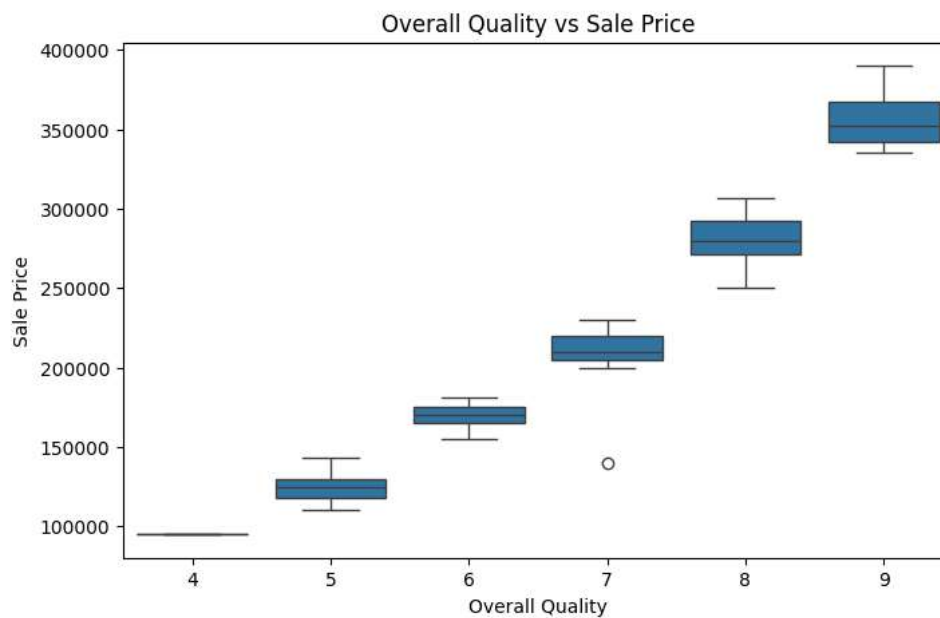
plt.title("Distribution of House Prices")
plt.xlabel("Sale Price")
plt.ylabel("Number of Houses")
plt.show()
```



```
plt.figure(figsize=(8,5))

sns.boxplot(x="OverallQual", y="SalePrice", data=df)

plt.title("Overall Quality vs Sale Price")
plt.xlabel("Overall Quality")
plt.ylabel("Sale Price")
plt.show()
```



```
plt.figure(figsize=(8,5))

sns.scatterplot(
    x="GrLivArea",
    y="SalePrice",
    data=df
)

plt.title("Living Area vs Sale Price")
plt.xlabel("Above Ground Living Area")
plt.ylabel("Sale Price")
plt.show()
```

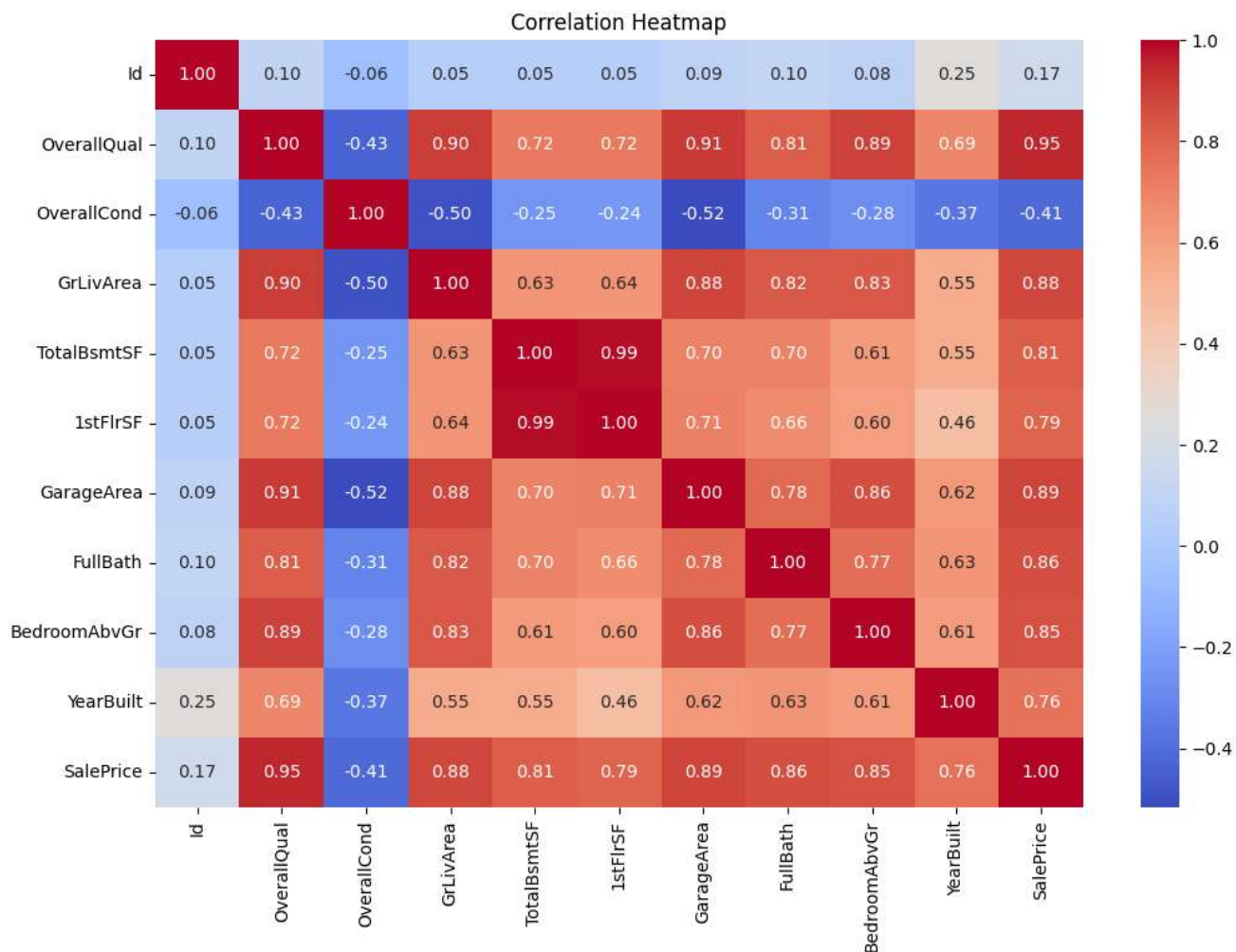


```
plt.figure(figsize=(12,8))

correlation = df.corr(numeric_only=True)

sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Heatmap")
plt.show()
```



```
correlation, p_value = pearsonr(
    df["GrLivArea"],
    df["SalePrice"]
)

print("Pearson Correlation:", correlation)
print("P-value:", p_value)
```

Pearson Correlation: 0.8781791196735441
P-value: 1.8160293741103668e-10

```
features = [
    "OverallQual",
    "GrLivArea",
    "TotalBsmtSF",
    "1stFlrSF",
    "GarageArea",
    "FullBath",
    "BedroomAbvGr",
    "YearBuilt"
]

X = df[features]
y = df["SalePrice"]

print("Features:")
print(X.head())

print("\nTarget:")
print(y.head())
```

Features:
OverallQual GrLivArea TotalBsmtSF 1stFlrSF GarageArea FullBath \

	OverallQual	GrLivArea	TotalBsmtSF	1stFlrSF	GarageArea	FullBath
0	7	1710	856	856	548	2

1	6	1262	1262	1262	460	2
2	7	1786	920	920	608	2
3	7	1717	756	961	642	1
4	8	2198	1145	1145	836	2

	BedroomAbvGr	YearBuilt
0	3	2003
1	3	1976
2	3	2001
3	3	1915
4	4	2000

Target:

0	208500
1	181500
2	223500
3	140000
4	250000

Name: SalePrice, dtype: int64

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

```
Training Data: (24, 8)
Testing Data: (6, 8)
```

```
model = LinearRegression()

model.fit(X_train, y_train)

print("Linear Regression Model Trained Successfully!")
```

Linear Regression Model Trained Successfully!

```
y_pred = model.predict(X_test)

print("Predicted House Prices:")
print(y_pred)
```

```
Predicted House Prices:
[105100.93978472 351066.36501401 281658.03274895 231146.10779086
 150527.46842741 95002.52311391]
```

```
mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, y_pred)

print("Mean Absolute Error (MAE):", mae)
print("Mean Squared Error (MSE):", mse)
print("Root Mean Squared Error (RMSE):", rmse)
print("R² Score:", r2)
```

```
Mean Absolute Error (MAE): 14490.952592778369
Mean Squared Error (MSE): 249481984.80463162
Root Mean Squared Error (RMSE): 15794.99872759196
R² Score: 0.9725130495075764
```

```
comparison = pd.DataFrame({
    "Actual Price": y_test.values,
    "Predicted Price": y_pred
})

print(comparison)
```

	Actual Price	Predicted Price
0	110000	105100.939785

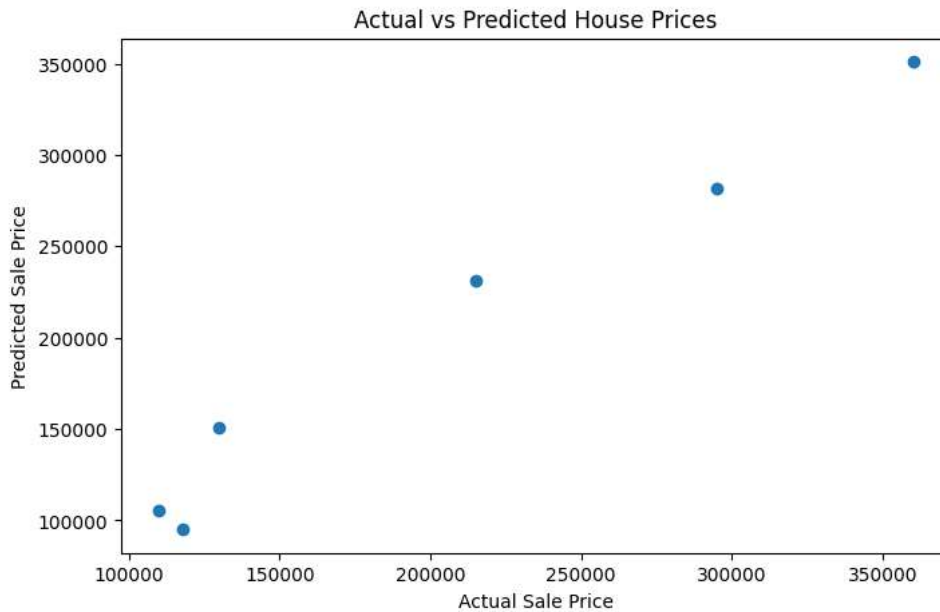
1	360000	351066.365014
2	295000	281658.032749
3	215000	231146.107791
4	129900	150527.468427
5	118000	95002.523114

```
plt.figure(figsize=(8,5))

plt.scatter(y_test, y_pred)

plt.xlabel("Actual Sale Price")
plt.ylabel("Predicted Sale Price")
plt.title("Actual vs Predicted House Prices")

plt.show()
```



```
coefficients = pd.DataFrame({
    "Feature": features,
    "Coefficient": model.coef_
})

print(coefficients.sort_values(
    by="Coefficient",
    ascending=False
))
```

	Feature	Coefficient
5	FullBath	24273.292649
0	OverallQual	17913.762976
7	YearBuilt	785.539152
3	1stFlrSF	148.508538
1	GrLivArea	37.717574
4	GarageArea	12.230073
2	TotalBsmntSF	-80.244255
6	BedroomAbvGr	-5350.614292

Q1. What are the major stages involved in an end-to-end data analytics project?

The major stages involved in an end-to-end data analytics project are:

1. **Data Collection** – Collecting data from reliable sources.
2. **Data Preprocessing** – Handling missing values, duplicate records, outliers and categorical data.
3. **Exploratory Data Analysis (EDA)** – Understanding patterns, relationships and trends in the dataset.
4. **Data Visualization** – Representing important information using graphs and charts.
5. **Statistical Analysis** – Finding correlations and performing statistical tests.
6. **Predictive Modeling** – Building machine learning models for prediction.
7. **Model Evaluation** – Evaluating the model using suitable performance metrics.

8. Business Insights – Converting analysis results into useful business recommendations.

Q2. Why is data preprocessing considered the foundation of analytics?

Data preprocessing is considered the foundation of analytics because real-world datasets may contain missing values, duplicate records, incorrect values and outliers. Preprocessing cleans and prepares the data before analysis.

Good-quality data helps in producing accurate analysis and reliable machine learning results. Therefore, proper preprocessing improves the quality and performance of the complete analytics process.

Q3. How does Exploratory Data Analysis (EDA) contribute to model development?

EDA helps in understanding the structure and characteristics of the dataset before building a machine learning model. It helps identify:

- Important variables
- Relationships between variables
- Trends and patterns
- Missing values
- Outliers
- Correlations

For example, in house price prediction, EDA can show the relationship between **living area**, **house quality** and **Sale Price**. This information helps in selecting useful features for the prediction model.

Q4. What factors should be considered while selecting a machine learning algorithm?

The following factors should be considered while selecting a machine learning algorithm:

1. Type of problem – classification, regression or clustering.
2. Size of the dataset.
3. Type of input features.
4. Relationship between variables.
5. Accuracy requirements.
6. Interpretability of the model.
7. Training and computational requirements.
8. Evaluation metrics suitable for the problem.

For the House Price Prediction project, **Linear Regression** is suitable because the target variable, SalePrice, is continuous.

Q5. Why is model evaluation important before deployment?

Model evaluation is important because it determines how well a machine learning model performs on unseen data. It helps identify whether the model provides reliable predictions.

For regression problems, metrics such as **MAE**, **MSE**, **RMSE** and **R² Score** can be used. A model should be evaluated before deployment to ensure that its predictions are sufficiently reliable for the intended application.

Q6. How do Business Intelligence (BI) tools complement machine learning?

Business Intelligence tools complement machine learning by presenting data and model results through interactive dashboards, charts and reports.

Machine learning focuses on prediction and pattern identification, while BI tools help users understand and communicate these results. Together, they help organizations monitor performance, identify trends and make data-driven decisions.

Q7. What are the major challenges in working with real-world datasets?

Major challenges include:

- Missing values
- Duplicate records
- Outliers
- Incorrect or inconsistent data
- Large dataset size
- Categorical variables
- Different data formats

- Data quality issues
- Feature selection
- Data privacy and security

These challenges need to be handled carefully during preprocessing and analysis.

Q8. How can data insights be converted into actionable business recommendations?

Data insights can be converted into actionable recommendations by identifying important patterns and connecting them with business objectives.

For example, in house price prediction, if analysis shows that **overall house quality and living area strongly influence Sale Price**, a business can use these factors while estimating property prices.

Thus, analytical findings should be translated into specific actions that support business planning and decision-making.

Q9. Why is presentation and data storytelling important in analytics?